BASES DE DATOS — PRÁCTICA FINAL

Ride-Hailing

GRUPO 12

David Sanez Camino, Miguel Ramírez Ochoa, Diego Vázquez Morales y Sofía Casado Sainz

Diseño e implementación de una base de datos relacional
para plataforma de transporte bajo demanda

MySQL 8.0 | Docker Compose | Prometheus | Grafana

ÍNDICE

01 Caso de uso y arquitectura

02 Diseño del esquema (MER)

03 Índices y rendimiento

04 Concurrencia y transacciones

05 Seguridad y permisos

06 Backup y recuperación

07 Dashboards y métricas

08 Demo

01 / CASO DE USO

¿Qué problema resuelve?

RIDER

Solicita viaje
A → B

→

SISTEMA

Genera oferta
y la distribuye

→

CONDUCTORES

Primer en aceptar
se queda el viaje

Estados del viaje:
solicitado → aceptado
→ en_curso → finalizado

Un conductor acepta,
el resto se rechazan
automáticamente

Historial de eventos
para auditoría
completa

Métricas en tiempo real
por conductor,
company y negocio

02 / DISEÑO DEL ESQUEMA

12 tablas · InnoDB · utf8mb4

usuario
Datos comunes de todos los usuarios

rider
Rol pasajero — método de pago

conductor
Rol conductor — licencia, company, rating

vehiculo
Vehículo asignado al conductor

company
Empresas operadoras (Cabify, Bolt…)

tarifa
Precio base + €/km + €/min por company

viaje
Entidad central — estado, geo, precio

oferta
Oferta generada por cada viaje

oferta_conductor
N:N oferta↔conductor + decisión

valoracion
Puntuación 1-5 tras finalizar viaje

evento_viaje
Log inmutable de cambios de estado

empresa_vehiculo
N:N company↔vehiculo con historial temporal

¿Por qué diseñamos así?

Separación usuario / rider / conductor

Usuario centraliza email, teléfono y estado. rider y conductor añaden solo atributos de rol. Evita duplicar campos comunes y facilita el login unificado.

Tarifa histórica con vigente_desde

Cada company puede actualizar precios sin afectar viajes ya realizados. El viaje referencia la tarifa vigente en el momento de su creación.

oferta_conductor: tabla intermedia N:N

Cuando se crea una oferta se inserta una fila por conductor con decision=pendiente. El trigger BEFORE UPDATE garantiza que nunca dos conductores acepten la misma oferta.

evento_viaje como log inmutable

Cada transición de estado inserta una nueva fila. Permite reconstruir la línea de tiempo de cualquier viaje y detectar anomalías sin modificar registros existentes.

empresa_vehiculo con PK compuesta

(id_company, id_vehiculo, fecha_asignacion) permite reasignaciones a lo largo del tiempo. fecha_fin=NULL indica asignación vigente.

valoracion con UNIQUE KEY (id_viaje)

Garantiza exactamente una valoración por viaje. CHECK constraint limita puntuación 1-5. Trigger actualiza valoracion_media del conductor en tiempo real.

03 / ÍNDICES Y RENDIMIENTO

idx_viaje_estado (viaje)
Filtros por estado (solicitado, en_curso...)

idx_viaje_rider (viaje)
Historial de viajes de un rider

idx_viaje_conductor_aceptado (viaje)
Búsquedas por conductor asignado

idx_viaje_creado_en (viaje)
Filtros y orden por fecha

idx_valoracion_conductor (valoracion)
Cálculo de valoración media

idx_valoracion_rider (valoracion)
Historial de valoraciones del rider

idx_vehiculo_conductor (vehiculo)
JOIN conductor → vehiculo

idx_evento_viaje (evento_viaje)
Timeline de eventos por viaje

Reglas de indexación

- Todas las FKs tienen índice — JOIN sin full-scan
- UNIQUE KEYs actúan como índice + restricción
- creado_en indexado para consultas de dashboard
- Índices nunca usados detectados en dashboard.sql
- performance_schema monitoriza IOPS por índice

Garantizar que solo un conductor acepta

TRIGGER BEFORE UPDATE

```
CREATE TRIGGER
  trg_oferta_conductor_unica_aceptacion
BEFORE UPDATE ON oferta_conductor
FOR EACH ROW
BEGIN
  IF NEW.decision = 'aceptada' THEN
    IF EXISTS (
      SELECT 1 FROM oferta_conductor
      WHERE id_oferta = NEW.id_oferta
        AND decision = 'aceptada'
        AND id_conductor <> NEW.id_conductor
    ) THEN
      SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
          'Ya existe un conductor que ha aceptado esta oferta';
    END IF;
  END IF;
END
```

FLUJO DE ACEPTACIÓN

1. START TRANSACTION
2. SELECT … FOR UPDATE (lock fila)
3. UPDATE oferta_conductor → aceptada
4. UPDATE oferta → aceptada
5. UPDATE viaje → aceptado + conductor_id
6. UPDATE resto conductores → rechazada
7. INSERT evento_viaje (auditoría)
8. COMMIT

05 / SEGURIDAD Y PERMISOS

app@%
Aplicación

- SELECT, INSERT, UPDATE en viaje, oferta, oferta_conductor
- SELECT, INSERT en rider
- SELECT en conductor, vehiculo, company

dashboard@%
Visualización

- SELECT en toda la BD
- SELECT en performance_schema
- Ver procesos activos
- Vista v_rider_publico (sin datos personales)

backup@localhost
Backup (local)

- SELECT + BLOQUEO DE TABLAS + VER VISTAS
- Solo acceso localhost
- Cuenta bloqueada cuando no se usa

admin@localhost
Administrador

- Todos los privilegios en rideHailing
- Solo acceso localhost
- Contraseña con expiración

ROLES: rol_lectura → dashboard | rol_escritura → app | Vista v_rider_publico oculta email y teléfono del rider

RPO = 1 hora · RTO = 4 horas

RPO
1 hora
Máx. pérdida de datos aceptable

RTO
4 horas
Tiempo máximo para restaurar

Tipo
Lógico
mysqldump — portabilidad total

Retención
5 días
Backups anteriores se eliminan solos

1 Programar con cron

```
0 * * * * bash backup.sh  (Linux)
while true; sleep 3600  (Windows/Git Bash)
```

2 Ejecutar mysqldump

```
--single-transaction --routines --triggers
--events  →  backup_YYYYMMDD_HHMMSS.sql
```

3 Verificar y rotar

```
Comprueba exit code 0
Borra backups > 5 días automáticamente
```

4 Restaurar

```
docker exec -i mysql mysql -uroot
-prootpass < backup_FECHA.sql
```

07 / DASHBOARDS Y MÉTRICAS

DASHBOARD TÉCNICO (BD)

- Conexiones activas vs. Max_connections
- Queries por tipo (SELECT, INSERT, UPDATE…)
- Buffer Pool Hit Ratio (objetivo > 95%)
- Row lock waits y deadlocks (debe ser 0)
- Slow queries ordenadas por SUM_TIMER_WAIT
- Índices no utilizados (count_star = 0)
- Transacciones abiertas en tiempo real

DASHBOARD DE NEGOCIO

- Viajes por hora del día (últimos 7 días)
- Distribución de viajes por estado
- Ofertas: aceptadas vs rechazadas vs pendientes
- Tasa de aceptación por conductor y company
- Ingresos totales por conductor y company
- €/km como métrica de eficiencia
- Top 10 conductores con más viajes finalizados

Grafana: Docker → localhost:3000 · Importar dashboard 14057 · Fuente: Prometheus (mysqld-exporter:9104)

GRACIAS